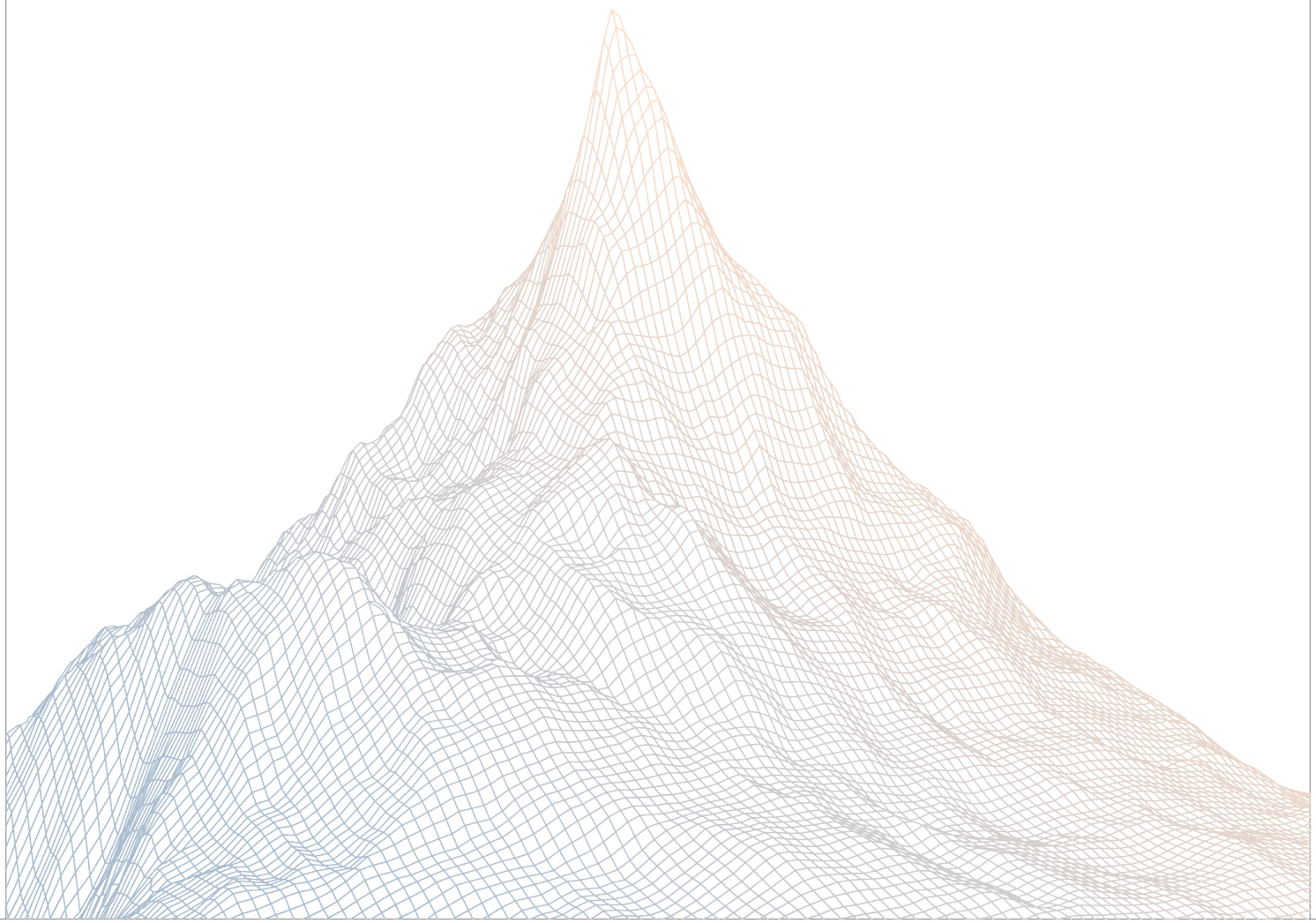
Zenith

Sage

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: March 23rd to March 30th, 2026

AUDITED BY: ether_sky
picotandc

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Sage 4

2.2 Scope 4

2.3 Audit Timeline 6

2.4 Issues Found 6

3 Findings Summary 6

4 Findings 8

4.1 Medium Risk 9

4.2 Low Risk 25

4.3 Informational 40

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Sage

Sage is a governed skill network for AI agents: a trust and distribution layer for agent capability.

Communities govern reusable skills and prompt libraries, publish the versions they trust, and give agents a machine-readable way to discover and use that canon.

The current product is already useful as a working library and governance system. The broader thesis is larger: as governed capability spreads across tools and agents, Sage can become economic infrastructure for how agent expertise is curated, distributed, and rewarded.

2.2 Scope

The engagement involved a review of the following targets:

Target	sage-audit
Repository	https://github.com/sage-protocol/sage-audit.git
Commit Hash	`4634947124876928dba5b2dbb4d39797ff2eecef`

Files	`SXXXProper.sol`
	`SimpleBountySystem.sol`
	`SageAuctionHouse.sol`
	`PromptGovernor.sol`
	`SubDAO.sol`

Target	Mitigation Review
Repository	https://github.com/sage-protocol/sage-audit.git
Commit Hash	`f6d75ca2889fa732b4660f5956a496386d8b2774`

Files	`SXXXProper.sol`
	`SimpleBountySystem.sol`
	`SageAuctionHouse.sol`
	`PromptGovernor.sol`
	`SubDAO.sol`

2.3 Audit Timeline

March 23, 2026	Audit start
March 30, 2026	Audit end
April 3, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	6
Low Risk	7
Informational	10
Total Issues	**23**

3

Findings Summary

ID	Description	Status
M-1	Racing updateTimelock execution ahead of queued proposals to orphan them on the old timelock	Acknowledged
M-2	Bounty creators cannot properly configure certain limitations	Resolved
M-3	The number of pending timelock operations is not tracked correctly in some cases	Resolved
M-4	Stranding active contributors behind a zero-return wall in getContributionLevelAt	Acknowledged
M-5	The deadline should be sufficiently far from the current time	Resolved
M-6	Anyone should be able to fork a prompt by burning SXXX tokens	Resolved
L-1	The addToWhitelist function can only be called when there are no submitters yet	Resolved
L-2	Stale subDAOFactory slot permanently disables fork paths on older SubDAOs	Resolved
L-3	quorum() resolves against a movable checkpoint instead of a per-proposal snapshot	Resolved
L-4	The emergency role can be accidentally revoked through the transferEmergencyPowers function	Resolved
L-5	The auction can be paused by malicious users	Acknowledged
L-6	The bounty creator can receive a keeper fee when finalizing the bounty	Resolved
L-7	The check in the _setMinBidIncrementBps function should be updated	Acknowledged
I-1	The snapshotBlock represents the block timestamp in the Bounty struct	Resolved
I-2	There is an incorrect comment in the _createBounty function	Resolved
I-3	There is no check for the SXXX token in the initialize function	Resolved

ID	Description	Status
I-4	The fee should be deducted from the reward when the bounty is cancelled	Acknowledged
I-5	The token balance check can be easily bypassed when submitting an entry	Acknowledged
I-6	The check for the CID length should be updated	Resolved
I-7	The factory deployer does not have a governance role	Resolved
I-8	The finalizeDeployment function should be called after initializeWithParams	Resolved
I-9	There is only one submitter in a DIRECT bounty	Acknowledged
I-10	The canSubmit function should return false once the deadline has passed	Acknowledged

4

Findings

4.1 Medium Risk

A total of 6 medium risk findings were identified.

[M-1] Racing `updateTimelock` execution ahead of queued proposals to orphan them on the old timelock

SEVERITY: Medium	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- PromptGovernor.sol
- PromptGovernor.sol

Description

A governance attacker (or a careless multisig) can swap the active timelock while another proposal is still queued on it — permanently stranding that proposal. The guard meant to prevent this has an off-by-one that the execution flow itself creates.

I traced the call path starting from `_executeOperations` at line 421. The function decrements `pendingTimelockOps` on lines 433—434, *then* calls `super._executeOperations` on line 437. That `super` call is what ultimately triggers the timelock to execute the batched calldata — including, in this scenario, `updateTimelock`. So by the time `_checkGovernance` runs its safety check, the executing proposal has already subtracted itself from the counter.

```
function _executeOperations(
    uint256 proposalId,
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) {
    // FIX #22: Decrement via operation-scoped accounting
    bytes32 opId = _timelockOperationId(proposalId, targets, values,
    calldatas, descriptionHash);
    address recorded = operationTimelock[opId];
    if (recorded ≠ address(0)) {
```

```
        delete operationTimelock[opId];
        if (pendingTimelockOps > 0) {
            pendingTimelockOps--;         // ⟵ decrements BEFORE super call
        }
    }
    super._executeOperations(proposalId, targets, values, calldatas,
    descriptionHash);
}
```

Walk through a concrete sequence. Proposal A (a routine treasury transfer) is queued — `pendingTimelockOps` goes to 1. Proposal B (carrying an `updateTimelock` call) is queued — counter goes to 2. Both sit in the timelock waiting for their delay. Proposal B's delay expires first, or both expire and someone simply executes B before A. During `_executeOperations` for Proposal B, `pendingTimelockOps` drops from 2 to 1 at line 434. Then `super._executeOperations` fires the batch, which hits `updateTimelock`, which calls `_checkGovernance`:

```
function _checkGovernance() internal override {
    super._checkGovernance();
    if (msg.sig == this.updateTimelock.selector) {
        require(pendingTimelockOps <= 1, "Drain queued ops before timelock
    swap");
    }
}
```

`pendingTimelockOps` is 1. The check is `≤ 1`. It passes. The timelock address is replaced.

Proposal A is now orphaned. It was scheduled on the old `TimelockController`, but the governor's `_executor()` now returns the new timelock address. `GovernorTimelockControl._executeOperations` will attempt to call `executeBatch` on the new timelock, which has no record of Proposal A's operation hash. The call reverts — or worse, if the old timelock held funds earmarked for A's execution, those funds are effectively locked (the old timelock has no way to release them without direct interaction, since the governor no longer points to it).

The root cause is straightforward. The `≤ 1` threshold assumes the executing proposal is still counted. It isn't — the decrement at line 434 runs before the governance check at line 494. So the guard tolerates exactly one extra pending operation when it should tolerate zero.

Recommendation

Change the threshold from `≤ 1` to `= 0`. After the self-decrement, the counter reflects only *other* pending operations. If any remain, the swap must be blocked:

```
function _checkGovernance() internal override {
    super._checkGovernance();
    if (msg.sig == this.updateTimelock.selector) {
        require(pendingTimelockOps <= 1, "Drain queued ops before timelock
            swap");
        require(pendingTimelockOps == 0, "Drain queued ops before timelock
            swap");
    }
}
```

Sage: Acknowledged. The active factory/deployer path uses PromptGovernorCloneable; rather than changing that path, we added proof coverage for callback decrement, single-op timelock swap, and queued-op rejection.

[M-2] Bounty creators cannot properly configure certain limitations

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- SimpleBountySystem.sol

Description:

When a `bounty` is created, values such as `minContributorLevel`, `minTokenBalance`, and `requiredBadgeId` are initialized to 0 by default.

```
function _createBounty(
    string memory title,
    string memory description,
    string memory ipfsCID,
    uint256 reward,
    uint256 deadline,
    uint256 votingDuration,
    BountyMode mode,
    LibraryAction libraryAction,
    address assignee,
    string memory libraryKey,
    bool useWhitelist
) internal returns (uint256) {

    bounties[bountyId] = Bounty({
        minContributorLevel: 0,  // Default to 0 (no restriction); creators
    can set via setGates()
        minTokenBalance: 0,      // Set via setGates() after creation
        requiredBadgeId: 0,      // Set via setGates() after creation
    });
}
```

`Bounty creators` can later configure these restrictions using the `setGates` and `setMinContributorLevel` functions. However, these functions can only be called when there are no submitters.

```
function setGates(
    uint256 bountyId,
    uint256 minTokenBalance,
    uint256 requiredBadgeId
) external {
    Bounty storage bounty = bounties[bountyId];
    _requireCreator(bounty);
    _requireActive(bounty);
    _requireNoSubmissions(bountyId);

    bounty.minTokenBalance = minTokenBalance;
    bounty.requiredBadgeId = requiredBadgeId;
}
```

As a result, if any user submits an entry before these functions are called, the creator will no longer be able to set these limitations.

Recommendations:

`Creators` should be able to set these values at the time of `bounty creation`, or there should be a defined period for configuring them before submissions are allowed.

Sage: Resolved with `@7196596178 ...`

Zenith: Verified.

[M-3] The number of pending timelock operations is not tracked correctly in some cases

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- PromptGovernor.sol

Description:

When `proposals` are queued, `pendingTimelockOps` is incremented.

```
function _queueOperations(
    uint256 proposalId,
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) returns (uint48) {
    unchecked {
        pendingTimelockOps++;
    }
}
```

These `queued proposals` can be executed or cancelled directly in the `timelock controller`. In the `timelock controller`, accounts with the `CANCELLER_ROLE` can cancel `queued proposals`, but such cancellations are not tracked in `PromptGovernor`.

```
function cancel(bytes32 id) public virtual onlyRole(CANCELLER_ROLE) {
    if (!isOperationPending(id)) {
        revert TimelockUnexpectedOperationState(
            id,
            _encodeStateBitmap(OperationState.Waiting) |
    _encodeStateBitmap(OperationState.Ready)
        );
    }
    delete _timestamps[id];
```

```
    emit Cancelled(id);
}
```

As a result, `pendingTimelockOps` does not decrease accordingly.

Even if `PromptGovernor` is set as the only entity with the `CANCELLER_ROLE`, the issue can still occur. For example, if the execution of a `queued proposal` reverts, `pendingTimelockOps` will not decrease.

```
function _executeOperations(
    uint256 proposalId,
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) {
    // FIX zenith-security/2026-03-sage-ether-sky#22: Decrement via
    operation-scoped accounting
    bytes32 opId = _timelockOperationId(proposalId, targets, values,
    calldatas, descriptionHash);
    address recorded = operationTimelock[opId];
    if (recorded ≠ address(0)) {
        delete operationTimelock[opId];
        if (pendingTimelockOps > 0) {
            pendingTimelockOps--;
        }
    }
    super._executeOperations(proposalId, targets, values, calldatas,
    descriptionHash);
}
```

This can lead to a situation where `pendingTimelockOps` never reaches zero, making it impossible to update the `timelock` permanently.

```
function _checkGovernance() internal override {
    super._checkGovernance();
    if (msg.sig == this.updateTimelock.selector) {
        require(pendingTimelockOps <= 1, "Drain queued ops before timelock
    swap");
    }
}
```

Recommendations:

Add a below function.

```
function checkCancelledProposal(
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) public returns (uint256) {
    uint256 proposalId = hashProposal(targets, values, calldatas,
    descriptionHash);

    require(state(proposalId) == ProposalState.Canceled, "");
    bytes32 opId = _timelockOperationId(proposalId, targets, values,
    calldatas, descriptionHash);
    address recorded = operationTimelock[opId];
    if (recorded != address(0)) {
        delete operationTimelock[opId];
        if (pendingTimelockOps > 0) {
            pendingTimelockOps--;
        }
    }

    return proposalId;
}
```

In that case, a trusted entity can directly cancel the reverted `proposal` in the `timelock controller` directly, and anyone can then call this function to correctly update `pendingTimelockOps`.

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[M-4] Stranding active contributors behind a zero-return wall in `getContributionLevelAt`

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- SimpleContributorSystem.sol
- SimpleBountySystem.sol

Description

A contributor who staked 5000 SXXX tokens six months ago and completed 12 tasks gets silently treated as a nobody — zero contribution level — the moment they try to submit to a gated bounty or vote on one. That's the practical outcome here. The system punishes exactly the users it should reward: long-standing participants who predate the checkpoint mechanism.

I traced `getContributionLevelAt` through `SimpleContributorSystem.sol` and hit the early return at line 107. The function checks `_contributionCheckpoints[user].length()` and bails with 0 if the user has no checkpoint history. This was an intentional fix (FIX #27) to prevent a different bug — legacy users inflating past contribution levels by leaking current state into historical queries. Fair enough as a standalone decision. But the caller in `SimpleBountySystem` doesn't know that.

```
function getContributionLevelAt(address user, uint256 timepoint)
    external view returns (uint256) {
    require(timepoint <= type(uint48).max, "timepoint-too-large");
    Checkpoints.Trace208 storage checkpoints
    = _contributionCheckpoints[user];
    Contributor storage c = contributors[user];
    // FIX #27: When no checkpoints exist, return 0 instead of using current
    state
    // as fallback. Using current state for historical queries inflates past
    contribution
    // levels for legacy/uninitialized users. Users can call
    checkpointContribution() to
    // seed their history and get accurate lookups going forward.
    if (checkpoints.length() == 0) {
        return 0;
```

```
    }
    uint48 key = uint48(timepoint);
    uint256 base = uint256(checkpoints.upperLookup(key));
```

Here's the disconnect. `SimpleBountySystem._getContributionLevelAtSnapshot` wraps the call in a try/catch — the `catch` branch falls back to `getContributionLevel(user)` (current state). The comment on line 1131 says *"fallback for older systems."* But `getContributionLevelAt` returning 0 doesn't revert. It succeeds. The catch never fires for legacy users; the try branch happily returns 0.

Two concrete paths break. First: bounty submission gating at line 400. If `bounty.minContributorLevel` is set to, say, 3, a legacy user with contribution level 8 (from real stake + activity) gets `level = 0`, fails the `level < minContributorLevel` check, and is blocked from submitting. Second: vote weighting at line 657. A legacy voter with 2000 voting power and contribution level 12 should get `rawMultiplier = sqrt(12) = 3`, yielding `weightedVotes = 2000 * 3 = 6000`. Instead they get `contributionLevel = 0`, which falls into the `contributionLevel > 1` false branch, so `rawMultiplier = 1` and `weightedVotes = 2000`. That's a 3x reduction in voting influence for an active participant.

The root cause is a design conflict between two contracts. FIX #27 in `SimpleContributorSystem` correctly prevents historical inflation. `SimpleBountySystem._getContributionLevelAtSnapshot` correctly assumes a fallback exists for legacy users (its own comment says so). Neither contract is wrong in isolation. The bug lives in the gap between them — a successful call returning 0 is indistinguishable from "this user genuinely has zero contribution."

Recommendation

The fix proposed in the finding is directionally correct. Modify the try branch in `_getContributionLevelAtSnapshot` to fall back when the historical lookup returns zero:

```
function _getContributionLevelAtSnapshot(address user, uint256 snapshot)
    internal
    view
    returns (uint256)
{
    // Use historical contribution snapshot where available; fallback for
    older systems.
    try contributorSystem.getContributionLevelAt(user, snapshot)
    returns (uint256 lvl) {
       return lvl;
       return lvl > 0 ? lvl : contributorSystem.getContributionLevel(user);
    } catch {
        return contributorSystem.getContributionLevel(user);
    }
```

```
}
```

One caveat worth noting: this partially reintroduces what FIX #27 was designed to prevent. A user with no checkpoint history but a current contribution level could claim that level for any past snapshot. In practice the risk is limited — `SimpleBountySystem` only queries snapshots set by bounty creators, not arbitrary past timepoints. A defense-in-depth alternative is to batch-call `checkpointContribution()` for all active legacy users via governance, seeding their history so the zero-return path stops triggering entirely.

Sage: Acknowledged. We did not retroactively alter historical bounty snapshots; instead, we added a forward-looking checkpoint backfill workflow for fresh deploys and operational recovery. Primary evidence lives in the helper/runbook rather than a standalone projected proof test.

[M-5] The deadline should be sufficiently far from the current time

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

The transaction to create the `bounty` may not be executed in time, leaving insufficient time for `submissions` as the `deadline` is reached too quickly.

```
function _createBounty(
    string memory title,
    string memory description,
    string memory ipfsCID,
    uint256 reward,
    uint256 deadline,
    uint256 votingDuration,
    BountyMode mode,
    LibraryAction libraryAction,
    address assignee,
    string memory libraryKey,
    bool useWhitelist
) internal returns (uint256) {
    require(deadline > block.timestamp, "deadline-future");

    bounties[bountyId] = Bounty({
        deadline: deadline,
    });
}
```

In such cases, `bounty creators` should be required to pay `expiration` or `cancellation fees`.

Recommendations:

```
function _createBounty(
    string memory title,
    string memory description,
    string memory ipfsCID,
    uint256 reward,
    uint256 deadline,
    uint256 minDuration,
    uint256 votingDuration,
    BountyMode mode,
    LibraryAction libraryAction,
    address assignee,
    string memory libraryKey,
    bool useWhitelist
) internal returns (uint256) {
    require(deadline > block.timestamp, "deadline-future");
    require(deadline > block.timestamp + minDuration, "deadline-future");
}
```

Sage: Resolved with @7196596178 ...

Zenith: Verified.

[M-6] Anyone should be able to fork a prompt by burning SXXX tokens

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- SubDAO.sol

Description:

In `PromptRegistry`, when `promptForkingPolicy` is set to `GOV_ONLY`, only accounts with `GOVERNANCE_ROLE` or `DEFAULT_ADMIN_ROLE` can call `addForkedPrompt`, while `MODERATOR_ROLE` cannot.

```
function addForkedPrompt(
    string calldata _originalCID,
    string calldata _newCID,
    string calldata _newName,
    string calldata _newDescription,
    address _forker
)
    external
    onlyRole(MODERATOR_ROLE)
    whenNotPaused
{
    if (promptForkingPolicy == ForkPolicy.GOV_ONLY) {
        // Only allow when called by governance (Timelock has
    DEFAULT_ADMIN_ROLE, Governor has GOVERNANCE_ROLE)
        require(hasRole(GOVERNANCE_ROLE, msg.sender) ||
    hasRole(DEFAULT_ADMIN_ROLE, msg.sender), "forking-gov-only");
    }
```

Since `SubDAO` holds the `GOVERNANCE_ROLE`, it is able to call `addForkedPrompt`. However, in the `_forkPromptWithBurn` function of `SubDAO`, normal users are prevented from forking prompts.

```
function _forkPromptWithBurn(
    string memory _originalCID,
    string memory _newCID,
```

```
    string memory _newName,
    string memory _newDescription
) internal {
    if (promptRegistry.promptForkingPolicy() ==
    PromptRegistry.ForkPolicy.GOV_ONLY) {
        if (!hasRole(GOVERNANCE_ROLE, msg.sender) &&
    !hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
            revert AccessControlUnauthorizedAccount(msg.sender,
    GOVERNANCE_ROLE);
        }
    }

    if (burnAmount > 0) {
        sxxxToken.burnFromForForking(msg.sender, burnAmount, _originalCID);
    }
```

This is inconsistent, as users are required to burn `SXXX tokens` to perform the fork. Therefore, users should be allowed to fork prompts through `SubDAO`, even when the policy is `GOV_ONLY`.

In other words, the `GOV_ONLY` restriction in `PromptRegistry` should not block user-initiated forking via `SubDAO` when it involves burning `SXXX tokens`.

Recommendations:

```
function _forkPromptWithBurn(
    string memory _originalCID,
    string memory _newCID,
    string memory _newName,
    string memory _newDescription
) internal {
    if (promptRegistry.promptForkingPolicy() == PromptRegistry.ForkPolicy.
        GOV_ONLY) {

    if (!hasRole(GOVERNANCE_ROLE, msg.sender) && !hasRole(DEFAULT_ADMIN_
            ROLE, msg.sender)) {
            revert AccessControlUnauthorizedAccount(msg.sender, GOVERNANCE_
                ROLE);
        }
    }
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

4.2 Low Risk

A total of 7 low risk findings were identified.

[L-1] The addToWhitelist function can only be called when there are no submitters yet

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

Currently, the `addToWhitelist` function can be called even when there are existing submitters.

```
function addToWhitelist(uint256 bountyId, address[] calldata addresses)
    external {
    Bounty storage bounty = bounties[bountyId];
    _requireCreator(bounty);
    _requireActive(bounty);
    require(addresses.length <= MAX_WHITELIST_BATCH, "Whitelist batch too
    large");
```

As a result, submitters who are not whitelisted may still be selected as winners.

Recommendations:

```
function addToWhitelist(uint256 bountyId, address[] calldata addresses)
    external {
    Bounty storage bounty = bounties[bountyId];
    _requireCreator(bounty);
    _requireActive(bounty);
    _requireNoSubmissions(bountyId);
```

```
    require(addresses.length <= MAX_WHITELIST_BATCH, "Whitelist batch too
    large");
```

Sage: Resolved with @7196596178 ...

Zenith: Verified.

[L-2] Stale `subDAOFactory` slot permanently disables fork paths on older SubDAOs

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- SubDAO.sol
- SXXXProper.sol

Description

An owner calling `SXXXProper.setSubDAOFactory` can silently strand every already-deployed SubDAO's fork capability — users who try to fork will burn 250 SXXX (or pay the stable fee) and then hit a revert on a factory contract that no longer exists. The root cause is a one-shot cache that never refreshes.

I traced `SubDAO.initialize` → `sxxxToken.getSubDAOFactory()` → the public `subDAOFactory` slot. At initialization the address is snapshot once into storage and never touched again. Both internal fork helpers — `_forkSubDAOWithBurn` and `_forkSubDAOWithStable` — read that same slot and cast it straight to `ISubDAOFactory` without re-querying the source of truth on `SXXXProper`. The slot declaration at line 98—99 in `SubDAO.sol` and the write at line 330 tell the whole story:

```
// SubDAO.sol — storage declaration (L98-99)
address public factoryDeployer;
address public subDAOFactory;

// ... inside initialize() (L325-330)
governor = PromptGovernor(p.governor);
promptRegistry = PromptRegistry(p.promptRegistry);
treasury = p.treasury;
timelock = p.timelock;
sxxxToken = SXXX(p.sxxxToken);
subDAOFactory = sxxxToken.getSubDAOFactory();   // cached once, forever
```

Meanwhile `SXXXProper` exposes a mutable setter (gated by `onlyOwner` and a `_factoryLocked` flag). Before the lock is engaged, the owner can call `setSubDAOFactory(newFactory)` freely — the three guards only prevent a zero address, a no-op, and a post-lock write. Suppose the protocol deploys 10 SubDAOs against factory A,

then rotates to factory B. Each of those 10 SubDAOs still holds address A in storage. If A is later self-destructed or simply stops registering new children, every `createForkedSubDAO` and `createForkedSubDAOWithStable` call reverts. The `_forkSubDAOWithStable` path is especially problematic because the factory fee query (`factory.prepareSubDAOForkFee`) also targets the stale address — so the revert may occur even before the actual fork call, making diagnosis harder.

In practice the blast radius is bounded by the `lockSubDAOFactory` function at `SXXXProper.sol` L218, which sets `_factoryLocked = true` irreversibly. Deployment runbooks already call this lock as a standard post-deploy step. Once locked, `setSubDAOFactory` reverts, so the cached pointer can never drift. The window of concern is deployment-to-lock only — roughly the first few transactions of a fresh deploy. If, say, 3 SubDAOs are created in that window and the factory pointer is rotated before `lockSubDAOFactory` executes, those 3 SubDAOs are permanently orphaned (there is no setter, refresher, or migration path on `SubDAO` for the `subDAOFactory` slot).

A numeric sketch: factory A is at `0xAAA...`, factory B at `0xBBB...`. SubDAO-1 stores `0xAAA...`. Owner calls `setSubDAOFactory(0xBBB...)`, then `lockSubDAOFactory()`. New SubDAOs (created via factory B) store `0xBBB...`. SubDAO-1 still holds `0xAAA...`. If `0xAAA...` has code size > 0 the fork may succeed against an obsolete contract; if code size == 0 the low-level call reverts. Either way the user's burn or stable fee is wasted or locked.

Recommendation

The simplest defence is to read the factory dynamically on every fork call instead of trusting the cached slot. Gas cost is one cold `SLOAD` on `SXXXProper` (≈ 2100 gas) — negligible next to the `burnFromForForking` and factory deployment calls in the same transaction.

```diff
-- a/contracts/SubDAO.sol
++ b/contracts/SubDAO.sol
@@ -696,7 +696,7 @@
         if (burnAmount > 0) {
             sxxxToken.burnFromForForking(msg.sender, burnAmount, name);
         }
        ISubDAOFactory factory = ISubDAOFactory(subDAOFactory);

    ISubDAOFactory factory = ISubDAOFactory(sxxxToken.getSubDAOFactory());
         factory.createForkedSubDAO(_newName, _newDescription, name,
    address(this), msg.sender, _maxForkFee, _copyLibrary);
         emit SubDAOForked(msg.sender, name, _newName, burnAmount);
     }
@@ -750,7 +750,7 @@
         ValidationLibrary.validateString(_newName, 1, 100);
         ValidationLibrary.validateString(_newDescription, 1, 200);
```

```
        ISubDAOFactory factory = ISubDAOFactory(subDAOFactory);

    ISubDAOFactory factory = ISubDAOFactory(sxxxToken.getSubDAOFactory());
        // Call the fee-based fork path on the factory; fee is collected
from msg.sender inside factory
        factory.createForkedSubDAOWithStable(
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[L-3] `quorum()` resolves against a movable checkpoint instead of a per-proposal snapshot

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- PromptGovernor.sol

Description

A governance majority that passes one `setQuorumVotes` proposal can retroactively lower the bar for a second proposal already sitting in its delay window — making it easier to entrench control once established. The root cause is an asymmetry: `PromptGovernor` pins the proposal threshold at creation but resolves quorum dynamically from a checkpoint timeline.

I traced `propose` → `proposalThresholdSnapshot[proposalId] = threshold` at L162 and confirmed this is a per-proposal freeze. Then I followed the quorum path: OZ's `_quorumReached` calls `quorum(proposalSnapshot(proposalId))`, which in this contract resolves to `quorum(voteStart - 1)`. The `quorum` function at L407—411 performs:

```
// PromptGovernor.sol L60-63 — storage
/// @notice Fixed quorum in absolute vote tokens (replaces percentage-based
    quorum)
uint256 public quorumVotes;
Checkpoints.Trace208 private _quorumVotesCheckpoints;

// L407-411 — dynamic resolution, not per-proposal
function quorum(uint256 timepoint) public view override(Governor)
    returns (uint256) {
    uint48 lookupTp = timepoint > type(uint48).max ? type(uint48).max :
    uint48(timepoint);
    uint208 q = _quorumVotesCheckpoints.upperLookupRecent(lookupTp);
    return uint256(q);
}
```

`upperLookupRecent` returns the most recent checkpoint keyed at or before `lookupTp`. Because `lookupTp` equals `voteStart - 1` (not proposal-creation time), any new checkpoint written into that gap shifts the quorum used when counting votes. Concretely: governance

calls `setQuorumVotes(newValue)` at L170, which pushes `_quorumVotesCheckpoints.push(clock(), ...)` at L175 — a fresh key that lands squarely inside the open window.

Walk through a concrete sequence. Quorum starts at 10 000 SXXX (10000e18). `votingDelay` is 2 days. At T_0 an operator creates proposal P_1. At T_0 + 26 hours a previously queued `setQuorumVotes(500e18)` executes via the timelock (the 1-day cooldown is satisfied since the last quorum change was > 24 h ago). P_1's snapshot resolves at T_0 + 2 days − 1 second; the checkpoint written at T_0 + 26 h is the most recent key ≤ that timestamp. So P_1 now needs only 500 SXXX to pass — a 20× reduction from the 10 000 SXXX the community expected when P_1 appeared on the governance dashboard. The operator needs 500 votes instead of 10 000, a gap of 9 500 SXXX.

Several guards soften the blow. `setQuorumVotes` is `onlyGovernance`, meaning the timelock must execute it — so an attacker already needs one successful proposal in the pipeline. The `quorumUpdateCooldown` of 1 day at L66/L172 throttles repeat changes. And `_MIN_QUORUM_VOTES = 500e18` at L54 prevents collapsing quorum to near-zero. These constraints make single-shot exploitation unlikely but do not prevent a patient governance majority from progressively ratcheting quorum downward across multiple proposal cycles (each cycle waits out the cooldown, each new proposal benefits from the latest drop).

Recommendation

Mirror what already exists for thresholds: freeze the quorum per-proposal at creation and use that frozen value when checking `_quorumReached`. One additional storage slot per proposal is cheap relative to the governance integrity it preserves.

```diff
-- a/contracts/PromptGovernor.sol
++ b/contracts/PromptGovernor.sol
@@ -50,6 +50,7 @@
     // Snapshot threshold at proposal creation for cancel-if-below checks
     mapping(uint256 => uint256) public proposalThresholdSnapshot;
    mapping(uint256 => uint256) public proposalQuorumSnapshot;

@@ -160,6 +161,7 @@
         proposalId = _propose(targets, values, calldatas, description,
    proposer);
         lastProposalTimestamp[proposer] = block.timestamp;
         proposalThresholdSnapshot[proposalId] = threshold;
        proposalQuorumSnapshot[proposalId] = quorum(clock());
         return proposalId;

@@ -407,7 +409,8 @@
     function quorum(uint256 timepoint) public view override(Governor)
    returns (uint256) {
```

```
uint48 lookupTp = timepoint > type(uint48).max ? type(uint48).max :
        uint48(timepoint);
    uint208 q = _quorumVotesCheckpoints.upperLookupRecent(lookupTp);
    return uint256(q);

// If called for a specific proposal via _quorumReached, the snapshot
    // override happens in _quorumReached itself; this fallback keeps
    // the public view functional for off-chain queries.

uint48 lookupTp = timepoint > type(uint48).max ? type(uint48).max :
        uint48(timepoint);

return uint256(_quorumVotesCheckpoints.upperLookupRecent(lookupTp));
 }
```

An `_quorumReached` override that reads `proposalQuorumSnapshot[proposalId]` instead of calling `quorum()` completes the fix. If modifying `_quorumReached` is undesirable (it is internal to OZ Governor), an alternative is to record `clock()` at creation and use it as the lookup key inside `quorum()` — though that requires mapping `timepoint → proposalId` in reverse.

Sage: Resolved with `@7584d55f64 ...`

Zenith: Verified.

[L-4] The emergency role can be accidentally revoked through the transferEmergencyPowers function

SEVERITY: Low	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- SubDAO.sol

Description:

If `newEmergencyAccount` is the same as `msg.sender`, the emergency role is effectively revoked.

```
function transferEmergencyPowers(address newEmergencyAccount)
    external onlyRole(EMERGENCY_ROLE) {
    if (newEmergencyAccount == address(0)) revert InvalidAccount();
    _grantRole(EMERGENCY_ROLE, newEmergencyAccount);
    _revokeRole(EMERGENCY_ROLE, msg.sender);
}
```

Recommendations:

```
function transferEmergencyPowers(address newEmergencyAccount)
    external onlyRole(EMERGENCY_ROLE) {
    if (newEmergencyAccount == address(0)) revert InvalidAccount();

    require(msg.sender ≠ newEmergencyAccount, "");

    _grantRole(EMERGENCY_ROLE, newEmergencyAccount);
    _revokeRole(EMERGENCY_ROLE, msg.sender);
}
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[L-5] The auction can be paused by malicious users

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- SageAuctionHouse.sol

Description:

In `VotingMultiplierNFT`, users can mint tokens via the `publicMint` function, which increases the minted amount for a given tier. Once this reaches `maxSupply`, any further minting will revert.

```
function _mintInternal(address to, uint256 tierId, string calldata uri)
    internal
    returns (uint256 tokenId)
{
    Tier storage tier = tiers[tierId];
    require(tier.multiplier >= 100, "Tier not found");
    require(tier.dao ≠ address(0), "Tier has no DAO");
    require(daoFactory.isSubDAO(tier.dao), "Not a registered SubDAO");
    require(tier.maxSupply == 0 || tier.minted < tier.maxSupply, "Tier sold
    out");
```

In `SageAuctionHouse`, if minting reverts, the auction is paused.

```
function _createAuction() internal {
    try nft.mint(address(this), mintTierId, defaultTokenURI)
    returns (uint256 nftId) {

    } catch {
        // If minting fails (e.g. paused), we pause the auction house
        _pause();
    }
}
```

When the owner attempts to `unpause`, `_createAuction` is called again, but this will continue to revert unless `mintTierId` is updated beforehand.

Recommendations:

At a minimum, the revert reason should be emitted so the `owner` can update `mintTierId` before `unpausing`.

Sage: Acknowledged. We kept the pause-on-mint-failure behavior, but documented and validated the recovery path: `unpause()` re-pauses until governance rotates mintTierId to a tier with remaining supply.

[L-6] The bounty creator can receive a keeper fee when finalizing the bounty

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- SimpleBountySystem.sol

Description:

The `bounty creator` does not receive the `keeper fee` in the `_expireAndRefund` function, but does receive it when finalizing the `bounty`.

```
function _expireAndRefund(
    Bounty storage bounty,
    uint256 bountyId,
    bool applyPenalty
) internal {
    uint256 penalty = applyPenalty ? (bounty.reward * EXPIRY_PENALTY_BPS)
    / 10000 : 0;
    if (penalty > 0) {
        uint256 keeperTip = (msg.sender ≠ bounty.creator)
            ? (penalty * KEEPER_FEE_BPS) / 10000
            : 0;
    }
}
```

Recommendations:

```
function finalizeCompetitive(uint256 bountyId) external nonReentrant {
    uint256 keeperFee = (bounty.reward * KEEPER_FEE_BPS) / 10000;
    uint256 keeperFee = msg.sender ≠ bounty.creator ? (bounty.reward *
        KEEPER_FEE_BPS) / 10000 : 0;

    address keeper = msg.sender ≠ bounty.creator ? msg.sender : address(0);
```

```
    _selectWinner(bountyId, winningId, msg.sender, keeperFee);
}
```

Sage: Resolved with @7196596178 ...

Zenith: Verified.

[L-7] The check in the _setMinBidIncrementBps function should be updated

SEVERITY: Low	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- SageAuctionHouse.sol

Description:

In the `_setMinBidIncrementBps` function, the following check was added to prevent infinite auction extensions.

```
function _setMinBidIncrementBps(uint256 newBps) internal {
    // Fix #23: Prevent zero increment (enables infinite auction extensions)
    require(newBps > 0, "Must be > 0");
}
```

However, this check is not sufficient.

Recommendations:

```
function _setMinBidIncrementBps(uint256 newBps) internal {
    // Fix #23: Prevent zero increment (enables infinite auction extensions)
    require(newBps > 0, "Must be > 0");
    require(newBps * reservePrice ≥ 10000, "");
}

function setReservePrice(uint256 _reservePrice) external onlyOwner {
    require(minBidIncrementPercentage * _reservePrice ≥ 10000, "");
    reservePrice = _reservePrice;
    emit AuctionReservePriceUpdated(_reservePrice);
}
```

Sage: Acknowledged. We did not add a new `minBidIncrement` / `reservePrice` invariant

because proof coverage showed the documented minimum operating range already rejects same-amount rebids.

4.3 Informational

A total of 10 informational findings were identified.

[I-1] The snapshotBlock represents the block timestamp in the Bounty struct

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

In the `Bounty` struct, the `snapshotBlock` is actually the block time not block number.

```
struct Bounty {
    uint256 snapshotBlock;       // Block number for voting power snapshot
}
```

Recommendations:

```
struct Bounty {
    uint256 snapshotBlock;       // Block number for voting power snapshot
    uint256 snapshotBlock;       // Block time for voting power snapshot
}
```

Sage: Resolved with @7196596178 ...

Zenith: Verified.

[I-2] There is an incorrect comment in the _createBounty function

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

Below comment is incorrect.

```
function _createBounty(
    string memory title,
    string memory description,
    string memory ipfsCID,
    uint256 reward,
    uint256 deadline,
    uint256 votingDuration,
    BountyMode mode,
    LibraryAction libraryAction,
    address assignee,
    string memory libraryKey,
    bool useWhitelist
) internal returns (uint256) {
    // Creator stake is currently configured in GovernanceConfig per SubDAO.
@-> // BPS is expressed as reward-relative stake (e.g., 150 = 150%).
    uint256 rewardStake = calculateStake(reward);
```

Recommendations:

```
function _createBounty(
    string memory title,
    string memory description,
    string memory ipfsCID,
    uint256 reward,
    uint256 deadline,
```

```
    uint256 votingDuration,
    BountyMode mode,
    LibraryAction libraryAction,
    address assignee,
    string memory libraryKey,
    bool useWhitelist
) internal returns (uint256) {
    // Creator stake is currently configured in GovernanceConfig per SubDAO.
    // BPS is expressed as reward-relative stake (e.g., 150 = 150%).
    // BPS is expressed as reward-relative stake (e.g., 150 = 1.5%).
    uint256 rewardStake = calculateStake(reward);
```

Sage: Resolved with `@7a78105056 ...` Stale/BPS comments were corrected during the struct API rewrite; this was documentation cleanup, not a runtime behavior change.

Zenith: Verified.

[I-3] There is no check for the SXXX token in the initialize function

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

In `SimpleBountySystem`, the check for the `SXXX token` is present in the constructor, but it is missing in the `initialize` function.

```
constructor(
    address _sxxxToken,
    address _contributorSystem,
    address _governanceConfig,
    address _communityTreasury,
    address admin,
    address _subdao
) {
    _requireTimestampClock(_sxxxToken);
```

Recommendations:

```
constructor(
    address _sxxxToken,
    address _contributorSystem,
    address _governanceConfig,
    address _communityTreasury,
    address admin,
    address _subdao
) {
    _requireTimestampClock(_sxxxToken);
}
```

```
function _initializeUnchecked(
    address _sxxxToken,
    address _contributorSystem,
    address _governanceConfig,
    address _communityTreasury,
    address admin,
    address _subdao
) internal {
    _requireTimestampClock(_sxxxToken);
}
```

Sage: Resolved with @7a78105056 ...

Zenith: Verified.

[I-4] The fee should be deducted from the reward when the bounty is cancelled

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

The `expiration fee` is calculated based on the `reward`, while the `cancellation fee` is applied to the creator's deposit.

Recommendations:

```
function cancelBounty(uint256 bountyId) external nonReentrant {
    // Charge 5% cancellation fee to treasury
    uint256 fee = (bounty.creatorDeposit * CANCELLATION_FEE_BPS) / 10000;
    uint256 fee = (bounty.reward * CANCELLATION_FEE_BPS) / 10000;
}
```

Sage: Acknowledged. We intentionally kept cancellation fees charged against `creatorDeposit` (reward + creator stake) and documented that behavior rather than changing the fee basis.

[I-5] The token balance check can be easily bypassed when submitting an entry

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

The `token balance check` can be easily bypassed by temporarily transferring tokens from other accounts.

```
function _passesGates(uint256 bountyId, address submitter)
    internal view returns (bool) {
    // Per-bounty token gate: enforce actual token balance, not delegated
    voting power.
    if (bounty.minTokenBalance > 0) {
        uint256 tokenBalance = sxxxToken.balanceOf(submitter);
        if (tokenBalance < bounty.minTokenBalance) {
            return false;
        }
    }
```

Recommendations:

This could be documented.

Sage: Acknowledged. The token gate remains a point-in-time `balanceOf()` check; we documented the temporary-transfer bypass limitation rather than redesigning the gate in this remediation set.

[I-6] The check for the CID length should be updated

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SubDAO.sol

Description:

In the `addForkedPrompt` function of `PromptRegistry`, the lengths of `_originalCID` and `_newCID` must not exceed 100.

```
unction addForkedPrompt(
    string calldata _originalCID,
    string calldata _newCID,
    string calldata _newName,
    string calldata _newDescription,
    address _forker
)
    external
    onlyRole(MODERATOR_ROLE)
    whenNotPaused
{
    require(bytes(_originalCID).length <= 100, "Original CID too long");
    require(bytes(_newCID).length <= 100, "New CID too long");
```

Therefore, the corresponding validation in the `_forkPromptWithBurn` function of `SubDAO` should be updated accordingly.

Recommendations:

```
function _forkPromptWithBurn(
    string memory _originalCID,
    string memory _newCID,
    string memory _newName,
    string memory _newDescription
) internal {
```

```
if (bytes(_originalCID).length == 0) revert SubDAOFactoryErrors.
    InvalidCIDRequired();

if (bytes(_newCID).length == 0) revert SubDAOFactoryErrors.InvalidCIDRequi
    red();
ValidationLibrary.validateString(_originalCID, 1, 100);
ValidationLibrary.validateString(_newCID, 1, 100);
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[I-7] The factory deployer does not have a governance role

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SubDAO.sol

Description:

The `factory deployer` calls `_initializeFromParams`, which grants `DEFAULT_ADMIN_ROLE` and `EMERGENCY_ROLE`.

```
function _initializeFromParams(SubDAOInitParams memory p) internal {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(EMERGENCY_ROLE, msg.sender);
}
```

However, in the `finalizeDeployment` function, it is treated as if it also has the `GOVERNANCE_ROLE`.

```
function finalizeDeployment() external {
    if (previousFactory ≠ address(0)) {
        _revokeRole(DEFAULT_ADMIN_ROLE, previousFactory);
        _revokeRole(EMERGENCY_ROLE, previousFactory);
        _revokeRole(GOVERNANCE_ROLE, previousFactory);
    }
}
```

Recommendations:

```
function _initializeFromParams(SubDAOInitParams memory p) internal {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(GOVERNANCE_ROLE, msg.sender);
    _grantRole(EMERGENCY_ROLE, msg.sender);
}
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[I-8] The finalizeDeployment function should be called after initializeWithParams

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- SubDAO.sol

Description:

If `finalizeDeployment` is called before `initializeWithParams`, the `initializeWithParams` call will revert.

Additionally, the `setFactoryDeployer` function should be executed immediately after the `subDAO` is created, which appears to be the case in the current deployment.

Recommendations:

```
function finalizeDeployment() external {
    require(initialized, "");
}
```

Sage: Resolved with `@2bd986afb5 ...`

Zenith: Verified.

[I-9] There is only one submitter in a DIRECT bounty

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

In a `DIRECT bounty`, there can only be one `submitter`. Once the `assignee` submits, the status changes to `CLAIMED`, preventing any further `submissions`.

```
function submitEntry(
    uint256 bountyId,
    string calldata promptIPFS,
    string calldata deliverableIPFS
) external nonReentrant returns (uint256) {
    require(bounty.status == BountyStatus.ACTIVE, "Bounty not active");

    // Mode-specific validation
    if (bounty.mode == BountyMode.DIRECT) {
        require(msg.sender == bounty.assignee, "Not the assignee");
        require(_passesGates(bountyId, msg.sender), "gate-fail");
    } else {
    }

    // Create submission
    uint256 submissionId = submissionCount[bountyId] + 1;

    if (bounty.mode == BountyMode.DIRECT) {
        bounty.winner = msg.sender;
        bounty.status = BountyStatus.CLAIMED;
        emit BountyClaimed(bountyId, msg.sender);
    }

    return submissionId;
}
```

Therefore, there is no need to explicitly pass a `submission ID` in the `approveSubmission` function.

Recommendations:

```
function approveSubmission(uint256 bountyId, uint256 submissionId)
function approveSubmission(uint256 bountyId)
    external
    nonReentrant
{
    Bounty storage bounty = bounties[bountyId];

    // Allow governance contract or admin role
    require(
        msg.sender == governanceContract || hasRole(GOVERNANCE_ROLE,
    msg.sender),
        "Not authorized"
    );
    require(bounty.status == BountyStatus.CLAIMED, "bad-status");

    Submission storage sub = submissions[bountyId][submissionId];
    require(sub.exists, "submission-missing");

    _selectWinner(bountyId, submissionId, address(0), 0);
    _selectWinner(bountyId, 1, address(0), 0);
}
```

Sage: Acknowledged. We intentionally kept the shared `approveSubmission(bountyId, submissionId)` entrypoint for DIRECT and COMPETITIVE flows; the DIRECT case is now documented as effectively using `submissionId == 1`.

[I-10] The canSubmit function should return false once the deadline has passed

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- SimpleBountySystem.sol

Description:

Users are only allowed to submit before the deadline, but this condition is not enforced in the `canSubmit` function.

```
function canSubmit(uint256 bountyId, address submitter)
    external view returns (bool) {
    Bounty storage bounty = bounties[bountyId];
    if (bounty.mode == BountyMode.DIRECT) {
        return submitter == bounty.assignee && _passesGates(bountyId,
    submitter);
    }
    return _passesGates(bountyId, submitter);
}
```

Recommendations:

```
function canSubmit(uint256 bountyId, address submitter)
    external view returns (bool) {
    Bounty storage bounty = bounties[bountyId];
     if (block.timestamp > bounty.deadline) return false;
    if (bounty.status ≠ BountyStatus.ACTIVE) return false;
    if (bounty.mode == BountyMode.DIRECT) {
        return submitter == bounty.assignee && _passesGates(bountyId,
    submitter);
    }
    return _passesGates(bountyId, submitter);
}
```

Sage: Acknowledged. `submitEntry()` remains the authoritative gate; we proved the helper mismatch for deadline, status, and duplicate submissions, but did not treat `canSubmit()` as a security-critical state transition.